

Deterministic MCP Guardrails for Safe Power-Pathway Design Agents

Chenrui Xu

Department of Civil and Environmental Engineering
Carnegie Mellon University
Pittsburgh, PA, USA
chenruix@andrew.cmu.edu

Abstract—Large language model (LLM) agents are increasingly used for safety-critical engineering decisions, yet they remain prone to arithmetic hallucination and constraint omission. We study whether deterministic, pre-execution guardrails implemented via the Model Context Protocol (MCP) can reduce invalid decisions in grid-constrained power-pathway planning. We introduce a 22-case benchmark spanning five failure modes (infeasible phases, budget violations, carbon violations, hallucination traps, and feasible baselines) and four agent conditions that progressively add structure: prompt-only (B1), self-critique (B2), tools without enforcement (B3), and MCP-guarded with mandatory validator calls (G1). Live evaluation with Gemini 2.5 Flash shows that providing validator tools without enforcement (B3) reduces the invalid-decision rate from 36.4% to 9.1%, while adding gateway enforcement (G1) further reduces it to 4.5%. Self-critique provides no improvement over the prompt-only baseline. In this benchmark, tool availability accounts for most of the improvement, with pre-execution enforcement adding a smaller but measurable safety margin.

Index Terms—LLM agents, guardrails, model context protocol, power systems, constraint enforcement, safe AI

I. INTRODUCTION

Data-center operators and grid planners are beginning to adopt LLM-based agents to recommend power-supply strategies, selecting among technologies such as grid interconnects, wind-plus-battery, solar-plus-battery, fuel cells, and small modular reactors [1]. These recommendations must satisfy hard engineering constraints: firm (dispatchable) capacity must cover peak load in every growth phase; total capital expenditure must fit within a budget ceiling; blended carbon intensity must not exceed a regulatory target; and system uptime must meet a reliability threshold.

Unfortunately, current LLMs are unreliable arithmetic reasoners [2], and their self-monitoring through chain-of-thought or self-critique is insufficient for safety-critical domains [3]. Even when a model “checks” its own work, it may reinforce incorrect reasoning rather than correct it.

The *Model Context Protocol* (MCP) [4] provides a standardized interface through which LLM agents call external tools. MCP gateways sit between the model and the tools, offering a natural enforcement point for pre-execution policy checks. Yet published MCP deployments focus on capability integration

(web search, code execution, database access) rather than safety enforcement.

Research question. *Do deterministic, pre-execution MCP guardrails reduce invalid-decision rates for LLM power-pathway planning agents, and what is the cost in tokens and latency?*

We make three contributions: (1) a 22-case benchmark (PowerPathway-22) with five failure modes and deterministic validator tools; (2) an MCP gateway that blocks final submission until prerequisite validators have been invoked; and (3) a live evaluation across 88 runs (4 agents $\times$ 22 cases) with Gemini 2.5 Flash. Tool availability reduces the invalid-decision rate from 36.4% to 9.1%, and mandatory enforcement further reduces it to 4.5%.

II. RELATED WORK

A. LLM Agents for Engineering

LLM agents have been applied to robotic design [5], chemical synthesis [6], and energy-system analysis [7]. These works highlight the ability of LLMs to reason qualitatively about engineering trade-offs but do not address the enforcement of hard numerical constraints. Schick et al. [8] introduced Toolformer, showing that LLMs can learn to call external APIs to improve factual accuracy; our work extends this paradigm by imposing *mandatory sequencing constraints* on which tools must be called before a final answer may be submitted. ReAct [9] introduced the paradigm of interleaving reasoning and tool-calling in LLM agents; our G1 agent follows this paradigm but adds a stateful enforcement layer that restricts which tools may be called and in what order.

B. Guardrails and Alignment

Constitutional AI [10] and RLHF train models to avoid undesirable outputs, but provide no formal guarantees at inference time. Output-filtering guardrails (e.g., NeMo-Guardrails [11], Guardrails AI [12]) post-process model text but cannot prevent the model from generating a hallucinated numerical answer that later slips through a lenient regex. Our approach is complementary: rather than filtering outputs, we enforce *preconditions on tool calls*, preventing the agent from finalizing

a recommendation before deterministic validators have cleared it.

Doshi et al. [13] argue that safety properties of tool-using agents should be enforced structurally at the tool boundary rather than through prompt discipline; our MCP gateway instantiates this principle. Yang et al. [14] show that prompts systematically underspecify intended behavior, motivating enforcement through typed tool permissions. Greshake et al. [15] demonstrate that indirect prompt injection can hijack agent workflows; we observe a related non-adversarial bypass where the model ignores tools entirely (Section IV-C).

C. Model Context Protocol

MCP [4] standardizes tool registration, schema validation, and result handling for LLM agents. Existing MCP servers expose capabilities (databases, web search, code sandboxes) but do not implement dependency or ordering enforcement. We extend the gateway pattern with a stateful *session* that tracks which validators have been called and blocks downstream calls until prerequisites are satisfied.

D. Power-Pathway Planning

Grid interconnect capacity, dispatch factors, and capital-cost models are well-characterized in the power-systems literature [16]. However, agent-based tools for power-pathway recommendation are nascent, and no prior work has evaluated LLM agents against a systematic constraint benchmark for this domain.

III. METHOD

A. Benchmark: PowerPathway-22

We construct 22 cases representing data-center or industrial facility planning scenarios with five failure modes (Table I). Each case specifies load phases (MW), confirmed grid capacity, budget ceiling, reliability target (% uptime), and carbon limit (kg CO₂/MWh). Technology parameters (capacity factor, capex/kW, carbon intensity, deployment time) are drawn from the NREL Annual Technology Baseline and the Power Pathway Optimizer prototype [17]. All cases and their ground-truth feasibility labels were constructed by the authors; the benchmark therefore primarily serves as a controlled testbed for the guardrail mechanism rather than a representative sample of real-world planning scenarios.

B. Deterministic Validator Tools

We implement each hard constraint as a deterministic validator tool. These validators serve two roles: (1) as *result invariants*, re-run post-hoc on every agent’s final supply mix to compute the invalid-decision metric; and (2) as *runtime tools* exposed to B3 and G1 during planning, where the difference is whether calling them is optional (B3) or enforced (G1).

The four validators check: **capacity**, firm (dispatchable) power $\sum_t CF_t \times MW_t \geq \text{peak_load}$, with grid capped at the confirmed interconnect; **cost**, total capex $\leq$ budget, using NREL ATB [16] cost rates; **carbon**, generation-weighted intensity $\leq$ target; and **reliability**, blended availability $\geq$

TABLE I
POWERPATHWAY-22 BENCHMARK CASES

Failure Mode	Count	Representative challenge
FEASIBLE	6	Positive control; valid mix exists
INFEASIBLE_PHASE	5	Load exceeds confirmed supply in some phase
BUDGET_VIOLATION	4	Cheapest valid mix exceeds budget ceiling
CARBON_VIOLATION	3	No allowed tech meets carbon target
HALLUCINATION_TRAP	4	Missing/ambiguous data tempts false assumption
Total	22	

```
# Simplified gateway logic
def call(tool_name, **kwargs):
    if tool_name not in TOOL_REGISTRY:
        return ToolResult("ERROR", "Unknown tool")
    if missing_args(tool_name, kwargs):
        return ToolResult("ERROR", "Missing required args")
    for condition, msg in BLOCKING_RULES[tool_name]:
        if condition(kwargs, session):
            return ToolResult("BLOCKED", msg)
    result = TOOL_REGISTRY[tool_name](**kwargs)
    session.record(tool_name, result)
    return result
```

Fig. 1. Simplified MCP gateway enforcement logic. Schema errors return ERROR; dependency violations return BLOCKED.

uptime target, with a backup bonus for fast-dispatch sources. For example, a 100MW solar-only proposal at 20% capacity factor contributes only 20MW of firm capacity, so it fails the capacity validator for a 100MW peak-load case even though its nameplate capacity appears sufficient. A fifth tool, **score_pathway**, computes a composite score from 0 to 100. All tools are pure functions with no side effects.

C. MCP Gateway

The MCP gateway (Fig. 1) maintains a per-run session object that records which validators have been invoked. A blocking rule prevents `submit_final_recommendation` from executing unless all four validators have been called. Attempting to bypass this check returns an ERROR status with an explanatory message, prompting the agent to complete the missing validators before resubmitting.

D. Agent Conditions

We compare four conditions that progressively add structure. B1 and B2 have no access to validator tools; B3 and G1 both have access to the same tools but differ in whether usage is enforced.

B1 (prompt-only) issues a single inference call with a system prompt describing the four constraint types and the required JSON output format. No tools are available.

B2 (self-critique) follows B1’s initial response with up to two rounds of structured critique prompts that explicitly enumerate each constraint, asking the model to check and revise.

B3 (tools, no enforcement) exposes the same six tools as G1 via the model’s native function-calling interface, and the system prompt instructs the agent to call all four validators before recommending. However, no gateway enforcement exists: the agent *may* submit a final recommendation without having called any validator. This condition isolates the effect of tool availability from the effect of mandatory enforcement.

G1 (MCP-guarded) is identical to B3 except that all tool calls pass through the MCP gateway, which blocks `submit_final_recommendation` unless all four validators have been called. This condition tests whether structural enforcement provides additional safety beyond tool availability with prompting.

E. Evaluation Metrics

We evaluate each of the 88 runs (4 agents $\times$ 22 cases):

- **invalid_decision**: the agent’s final decision is invalid if it either submits a supply mix that violates ≥ 1 hard constraint (capacity, cost, carbon, or reliability), as determined by re-running all four deterministic validators, or incorrectly declares a feasible case infeasible/unknown. For G1, the gateway structurally prevents submission before validators complete, so remaining invalid decisions measure whether the *agent correctly interprets validator feedback and identifies feasibility*, not merely whether it called the tools.
- **correct_feasibility**: the agent correctly identified whether the case is feasible or infeasible (or unknown for HT-002). This is the primary metric of *reasoning quality*.
- **validators_called** (B3 and G1): number of the four required validators invoked per run.
- **total_tokens** and **latency_sec**: efficiency.

IV. RESULTS

A. Overall Safety Metrics

Table II summarizes the live evaluation results across all 88 runs (4 agents $\times$ 22 cases, Gemini 2.5 Flash). G1 reduces the invalid-decision rate to 4.5% (1/22), compared to 36.4% (8/22) for both B1 and B2. Notably, self-critique (B2) provides *no improvement* over the prompt-only baseline: both produce 8 invalid decisions across identical failure modes. B3, which has access to the same tools as G1 but without gateway enforcement, achieves 9.1% (2/22), a substantial improvement over B1/B2 but still twice G1’s rate. The comparison indicates that tool availability helps, while enforcement may provide an additional safety margin. The *correct feasibility* metric shows a similar ordering: B1 59.1%, B2 61.9%, B3 85.7%, G1 90.0%. Token overhead for B3 and G1 is 21.8k and 26.1k tokens respectively, reflecting multi-round function-calling, but latency remains modest (41 s).

B. Breakdown by Failure Mode

Table III shows invalid-decision counts by failure mode. The most striking pattern is the FEASIBLE regime: B1 and B2 each produce 5/6 invalid decisions: they claim infeasibility for cases where a valid supply mix exists, unable to find

TABLE II
AGENT COMPARISON ACROSS 22 CASES (LIVE, GEMINI 2.5 FLASH)

Agent	Invalid%	Correct%	Avg Tok	Avg Lat (s)	Validators
B1	36.4	59.1	1,161	34.5	N/A
B2	36.4	61.9	8,589	87.6	N/A
B3	9.1	85.7	21,783	41.1	3.86
G1	4.5	90.0	26,067	41.7	3.82

Invalid% denotes invalid final decisions, including both constraint-violating mixes and incorrect infeasibility/unknown decisions. Correct% excludes runs with no parseable feasibility decision (denominators: B1=22, B2=21, B3=21, G1=20).

TABLE III
INVALID DECISIONS BY FAILURE MODE (COUNT/TOTAL, LIVE)

Failure Mode	B1	B2	B3	G1
FEASIBLE	5/6	5/6	2/6	1/6
INFEASIBLE_PHASE	1/5	1/5	0/5	0/5
BUDGET_VIOLATION	0/4	0/4	0/4	0/4
CARBON_VIOLATION	0/3	0/3	0/3	0/3
HALLUCINATION_TRAP	2/4	2/4	0/4	0/4
Total	8/22	8/22	2/22	1/22

one without validator guidance. B3 reduces this to 2/6 and G1 to 1/6, confirming that validator tools help the model discover feasible mixes that it cannot find through reasoning alone. All four agents handle BUDGET_VIOLATION and CARBON_VIOLATION correctly (0 invalids), because these cases are infeasible and the models correctly decline to propose a mix. On HALLUCINATION_TRAP cases, B1 and B2 each produce 2/4 invalid decisions; both B3 and G1 produce 0/4, as validator feedback exposes the missing or inconsistent data before submission.

C. Tool Usage Patterns in B3 and G1

Both B3 and G1 called validators at similar rates: B3 averaged 3.86 validators per run and G1 averaged 3.82. When tools are available and the prompt instructs their use, the model generally complies, even without enforcement. The difference between B3 and G1 is therefore not in *whether* validators are called, but in how the model responds to validator feedback. B3’s 2 invalid decisions both occurred in FEASIBLE cases despite calling all four validators: in one case (F-001) the model iterated repeatedly but could not converge on a capacity-satisfying mix; in another (F-003) it incorrectly concluded the case was infeasible. G1 produced one similar error, suggesting that enforcement helps the model iterate more carefully but does not fully eliminate reasoning failures.

In one G1 run, the model returned a plain-text response without invoking any tools; this is a known limitation addressable at the agent level by requiring tool-call output format.

V. DISCUSSION AND THREATS TO VALIDITY

Construct validity. The benchmark cases and ground-truth labels were constructed by the same author who implemented the guardrails, creating a risk of confirmation bias in both case design and metric definition. Independent validation of the

benchmark, for example by domain experts or via crowdsourced annotation, is needed to establish external validity.

Internal validity. The benchmark spans five failure modes but remains small (22 cases). It may not cover all failure patterns encountered in real deployments. The deterministic validators are simplified models that do not capture real-world grid dynamics (e.g., transmission constraints, curtailment, market pricing).

External validity. All results use Gemini 2.5 Flash at default temperature. Behavior may differ for other model families (GPT-4o, Claude 3.5, Llama 3) or under non-default temperature settings. Real agents face noisier, less structured case descriptions than our benchmark provides.

Cost and benefit. B3 and G1 use 21.8k and 26.1k tokens per run ($18.8\times$ and $22.5\times$ B1), primarily from multi-round function calls and model thinking. However, latency for both is ~ 41 s, only $1.2\times$ B1 and actually faster than B2 (87.6 s), since sequential critique rounds are expensive. For 100+ MW infrastructure decisions with capital expenditure in the hundreds of millions, the token overhead is justified given the 4 to $8\times$ reduction in invalid decisions.

Alternative designs. A lighter-weight alternative to gateway enforcement is post-hoc repair: run the agent without safeguards, re-run the validators on its proposed mix, and feed any violations back as error messages for the agent to revise on a subsequent turn. This avoids MCP-level modification and may match G1 on benchmarks where validators are pure functions and a wrong intermediate answer is recoverable. Pre-execution enforcement remains preferable when tool calls carry side effects (e.g., procurement, dispatch commitments) or when the agent’s first emitted recommendation is itself acted upon, which is the regime our gateway targets. We therefore do not claim that pre-execution enforcement dominates post-hoc repair; our comparison isolates enforcement against tool availability without mandatory enforcement. A head-to-head comparison of pre-execution enforcement versus post-hoc repair under matched token budgets is left to future work.

Generalizability. The core mechanism, a stateful gateway enforcing prerequisite tool calls, should generalize to other safety-critical agentic domains (structural design, chemical process design, medical treatment planning). Future work should evaluate whether the architecture transfers without domain-specific tuning.

VI. CONCLUSION

This paper presents a deterministic MCP guardrail framework for LLM-based power-pathway planning agents, evaluated in a live experiment with Gemini 2.5 Flash across 88 runs (4 agents $\times$ 22 cases). The primary finding is that providing validator tools (B3) reduces the invalid-decision rate from 36.4% to 9.1%, and adding mandatory pre-execution enforcement (G1) further reduces it to 4.5%. Self-critique (B2) provides no benefit over the prompt-only baseline. Tool availability accounts for most of the observed improvement, while enforcement can provide an additional safety margin for constraint-heavy engineering domains. Future work will extend the benchmark, evaluate

across multiple model families, and investigate adversarial attacks targeting gateway bypass.

REFERENCES

- [1] International Energy Agency, “Electricity 2024: Data centres and artificial intelligence,” <https://www.iea.org/reports/electricity-2024>, 2024.
- [2] S. Frieder, L. Pinchetti, R.-R. Griffiths, T. Salvatori, T. Lukasiewicz, P. Petersen, and J. Berner, “Mathematical capabilities of ChatGPT,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [3] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, “Large language models cannot self-correct reasoning yet,” *arXiv preprint arXiv:2310.01798*, 2023.
- [4] Anthropic, “Model context protocol,” <https://modelcontextprotocol.io>, 2024.
- [5] F. Stella, C. Della Santina, and J. Hughes, “How can LLMs transform the robotic design process?” *Nature Machine Intelligence*, vol. 5, pp. 561–564, 2023.
- [6] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, “Autonomous chemical research with large language models,” *Nature*, vol. 624, 2023.
- [7] X. Zheng, Z. Wang, and Y. Tang, “Large language models for energy systems planning: Opportunities and challenges,” *Applied Energy*, vol. 364, p. 123116, 2024.
- [8] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [10] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, and others, “Constitutional AI: Harmlessness from AI feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [11] T. Rebeadea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, “NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails,” in *Proceedings of EMNLP (System Demonstrations)*, 2023.
- [12] Guardrails AI, “Guardrails AI: Adding guardrails to large language models,” <https://guardrailsai.com>, 2023.
- [13] A. Doshi, Y. Hong, C. Xu, E. Kang, A. Kapravelos, and C. Kästner, “Towards verifiably safe tool use for LLM agents,” in *Proceedings of the International Conference on Software Engineering, New Ideas and Emerging Results (ICSE-NIER)*, 2026.
- [14] C. Yang, Y. Shi, Q. Ma, M. X. Liu, C. Kästner, and T. Wu, “What prompts don’t say: Understanding and managing underspecification in LLM prompts,” *arXiv preprint arXiv:2505.13360*, 2025.
- [15] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [16] National Renewable Energy Laboratory, “Annual technology baseline 2023,” <https://atb.nrel.gov>, 2023.
- [17] C. Xu, “Power pathway optimizer: Interactive tool for data-center energy planning,” <https://eric-xc.github.io/power-pathway-optimizer/>, 2025.